DESENVOLVIMENTO MOBILE · MÓDULO 09 · COMUM ÀS TRILHAS · NÍVEL INTERMEDIÁRIO

Fundamentos de Banco de Dados

Modelagem, modelo relacional, normalização, SQL com SQLite, transações, índices, NoSQL e segurança

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o que é um banco de dados, um SGBD e os principais tipos de banco.
- Modelar um problema com entidades, atributos e relacionamentos (diagrama ER).
- Converter o modelo para tabelas com chaves primárias, estrangeiras e restrições.
- Normalizar tabelas até a 3ª forma normal e saber quando desnormalizar.
- Escrever SQL no SQLite: criar tabelas, inserir, alterar, apagar e consultar com filtros, agregações e *joins*.
- Entender transações, índices, bancos NoSQL e como evitar *SQL injection*.

Por que antes do Room?

O Room (Módulo 10) gera e verifica SQL para você, mas não pensa por você: tabelas mal modeladas e consultas mal escritas continuam lentas e confusas.

Roteiro

Neste módulo

- Conceitos
- Modelagem conceitual
- Modelo relacional
- Normalização
- SQL: definindo e alterando dados
- SQL: consultas
- Transações e índices
- Bancos NoSQL
- Segurança e prática
- Fechamento

Conceitos

Dados, banco de dados e SGBD

- **Banco de dados**: coleção organizada de dados relacionados, com estrutura definida.
- **SGBD** (Sistema Gerenciador de Banco de Dados): software que armazena, consulta e protege esses dados. Ex.: SQLite, PostgreSQL, MySQL, Firestore.
- O app nunca mexe no arquivo diretamente: ele pede ao SGBD, normalmente em **SQL**.

Por que não um arquivo de texto? Com milhares de leituras de sensor, um banco permite **buscar rápido**, **evitar duplicação**, **garantir integridade** e **não corromper dados** se o app fechar no meio da gravação.



SQLite

SGBD relacional **embutido**: uma biblioteca dentro do app, sem servidor, que guarda tudo em um único arquivo. Vem em todo Android e iOS e é a base do Room (Kotlin) e do Drift e do sqflite (Flutter).

Tipos de banco de dados

Tipo	Como organiza	Exemplos	Uso típico
Relacional (SQL)	tabelas com linhas e colunas, ligadas por chaves	SQLite, PostgreSQL, MySQL	dados estruturados, consultas flexíveis, integridade
Documentos	documentos JSON agrupados em coleções	Firestore, MongoDB	dados hierárquicos, sincronização em tempo real
Chave-valor	pares chave → valor	DataStore, Redis	preferências, cache, sessões
Séries temporais	medições indexadas pelo tempo	InfluxDB, TimescaleDB	telemetria de sensores em grande volume
Grafos	nós e arestas	Neo4j	redes sociais, recomendações

No LabMonitor: **SQLite** no aparelho (Room ou Drift, Módulo 10), **chave-valor** para preferências, **Firestore** (Módulo 12) e **PostgreSQL** (Módulos 16 e 17) na nuvem.

Por dentro de um SGBD



O caminho de uma consulta

O SQL é analisado, o **otimizador** escolhe o plano mais barato (usar um índice ou varrer a tabela?) e o executor lê as páginas necessárias. O gerenciador de transações garante ACID; o de armazenamento organiza os dados em páginas.

Um pouco de história

Edgar F. Codd, da IBM, propôs o modelo relacional em 1970. A linguagem SEQUEL, depois SQL, surgiu na IBM em 1974 e virou padrão ANSI em 1986. O SQLite foi criado por D. Richard Hipp em 2000.

Modelagem conceitual

Entidades, atributos e relacionamentos

- **Entidade**: "coisa" sobre a qual guardamos dados (Bancada, Equipamento, Usuário).
- **Atributo**: característica da entidade (nome, sala, limite de temperatura).
- **Identificador**: atributo que distingue cada ocorrência (número de patrimônio).
- **Relacionamento**: associação entre entidades (uma bancada *possui* equipamentos).
- **Cardinalidade**: quantos de cada lado participam: **1:1**, **1:N** ou **N:N**.

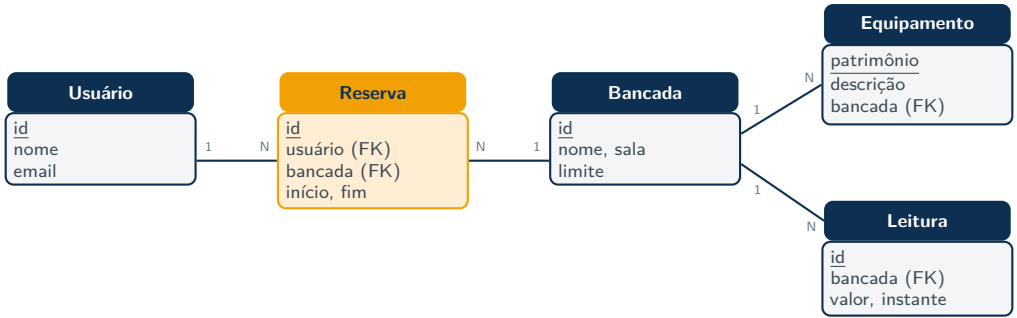
Como descobrir as entidades

Descreva o problema em frases e procure os **substantivos**:

"Um **usuário** faz **reservas** de **bancadas**. Cada bancada tem vários **equipamentos** e um **sensor** que gera **leituras** de temperatura."

Os **verbos** costumam virar relacionamentos (*faz*, *tem*, *gera*) e os **adjetivos** e medidas, atributos.

Diagrama ER do LabMonitor



Usuário N:N Bancada: a entidade **Reserva** resolve esse relacionamento e guarda seus próprios atributos (início, fim).

Ferramentas

draw.io, dbdiagram.io ou brModelo (muito usado no Brasil).

Modelo relacional

Tabelas, chaves e restrições

tabela `bancadas`

id	nome	sala	limite
1	Bancada 1	L-12	30.0
3	Bancada 3	L-12	32.0
4	Bancada 4	L-14	30.0

tabela `equipamentos`

patrimonio	descricao	bancada_id
IFMA-0042	Osciloscópio	3
IFMA-0107	Fonte simétrica	3
IFMA-0211	Gerador de funções	1

- **Tabela** (relação), **linha** (registro) e **coluna** (atributo).
- **Chave primária (PK)**: identifica cada linha; única e não nula.
- **Chave estrangeira (FK)**: coluna que aponta para a PK de outra tabela (`bancada_id` → `bancadas.id`).
- **Restrições**: `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.

A FK garante a **integridade referencial**: não existe equipamento numa bancada que não existe.

Do diagrama ER para tabelas

1:N

A tabela do lado **N** recebe uma FK para o lado **1**.

```
equipamentos.bancada_id
leituras.bancada_id
```

N:N

Cria-se uma **tabela associativa** com uma FK para cada lado.

```
reservas(usuario_id,
  bancada_id, inicio, fim)
```

1:1

A FK vai em qualquer um dos lados, com `UNIQUE`; muitas vezes as duas entidades viram uma só tabela.

Tipo SQLite	Guarda	No LabMonitor
`INTEGER`	inteiros de até 8 bytes	ids, `instante` em milissegundos (*epoch*)
`REAL`	ponto flutuante de 8 bytes	temperatura, limite
`TEXT`	texto UTF-8	nome, sala, patrimônio
`BLOB`	bytes brutos	raramente: prefira guardar arquivos fora do banco
`NULL`	ausência de valor	leitura ainda não recebida

Dica

O SQLite não tem tipo de data nem booleano: use `INTEGER` (*epoch* em ms; 0/1) e converta no app.

A base matemática: álgebra relacional

Operação	Símbolo	Em SQL
Seleção (filtra linhas)	σ	`WHERE`
Projeção (escolhe colunas)	π	lista do `SELECT`
Junção	$\bowtie$	`JOIN ... ON`
Produto cartesiano	$\times$	`CROSS JOIN`
União	$\cup$	`UNION`
Diferença	$-$	`EXCEPT`

Álgebra relacional

Conjunto de operações sobre relações (tabelas) que produzem novas relações. É a teoria por trás do SQL e do otimizador de consultas.

Nome das bancadas livres da sala L-12

$$\pi_{\text{nome}}\left(\sigma_{\text{sala}=\text{'L-12'} \wedge \text{livre}=1}(\text{bancadas})\right)$$

```
SELECT nome FROM bancadas
WHERE sala = 'L-12' AND livre = 1;
```

Por que isso importa

Como as operações têm propriedades algébricas (por exemplo, filtrar antes de juntar dá o mesmo resultado), o otimizador pode reescrever a consulta numa forma equivalente e muito mais rápida.

Normalização

O problema da “planilha única”

reserva	usuário	e-mail	bancada	sala	equipamentos
1	Ana	ana@ifma.edu.br	Bancada 3	L-12	Osciloscópio, Fonte
2	Ana	ana.s@ifma.edu.br	Bancada 1	L-12	Gerador
3	Bruno	bruno@ifma.edu.br	Bancada 3	L-14	Osciloscópio, Fonte

Anomalia de atualização

O e-mail da Ana está repetido e já divergiu. Qual é o certo?

Anomalia de inserção

Não dá para cadastrar uma bancada nova sem inventar uma reserva.

Anomalia de exclusão

Apagar a única reserva do Bruno apaga também o e-mail dele.

Normalizar é dividir os dados em tabelas de modo que **cada fato fique guardado uma única vez**.

As três primeiras formas normais

1ª forma normal (1FN)

Cada célula guarda **um único valor** atômico.

"Osciloscópio, Fonte" viola a 1FN → uma linha por equipamento, em tabela própria.

2ª forma normal (2FN)

Está na 1FN e todo atributo depende da **chave inteira**, não de parte dela.

Em `(reserva, equipamento)`, a data depende só da reserva → separar.

3ª forma normal (3FN)

Está na 2FN e nenhum atributo depende de **outro atributo não-chave**.

`sala` depende de `bancada`, não da reserva → vai para `bancadas`.



Regra de bolso da 3FN

"Cada atributo depende da chave, da chave inteira e de nada além da chave."

Quando desnormalizar

Normalização evita inconsistência, mas exige *joins* para remontar a informação. Às vezes vale **duplicar de propósito**.

Desnormalizações aceitáveis

- Guardar `ultima_temperatura` na tabela `bancadas` para o painel abrir instantaneamente.
- Cache local de dados vindos da API em uma tabela "achatada".
- Em bancos de documentos (Firestore), copiar o nome da bancada dentro da reserva.

Condições

- Exista uma **fonte da verdade** clara (a tabela normalizada).
- A cópia seja atualizada junto, na mesma transação, ou aceite ficar levemente defasada.
- O ganho seja medido, não suposto.

Dica

Regra prática: **normalize primeiro**, desnormalize depois e só onde a consulta realmente pesar.

SQL: definindo e alterando dados

DDL: criando as tabelas

labmonitor.sql

```sql
PRAGMA foreign_keys = ON;   -- liga a checagem de FKs
CREATE TABLE bancadas (
  id      INTEGER PRIMARY KEY,
  nome    TEXT    NOT NULL UNIQUE,
  sala    TEXT    NOT NULL,
  limite  REAL    NOT NULL DEFAULT 30.0
          CHECK (limite BETWEEN 0 AND 100)
);
CREATE TABLE leituras (
  id         INTEGER PRIMARY KEY,
  bancada_id INTEGER NOT NULL
             REFERENCES bancadas(id) ON DELETE CASCADE,
  valor      REAL    NOT NULL,
  instante   INTEGER NOT NULL          -- epoch em ms
);
CREATE INDEX idx_leituras_bancada_instante
  ON leituras (bancada_id, instante);
ALTER TABLE bancadas ADD COLUMN responsavel TEXT;
```

DDL — *Data Definition Language*

Define a estrutura: `CREATE`, `ALTER` e `DROP` de tabelas, índices e visões.

`INTEGER PRIMARY KEY`

No SQLite, essa coluna vira o *rowid*: se você não informar o valor, ele é gerado automaticamente.

Chaves estrangeiras no SQLite

Só são verificadas com `PRAGMA foreign_keys = ON` em cada conexão.

DML: inserindo, alterando e apagando

Manipulando dados

```sql
INSERT INTO bancadas (id, nome, sala, limite)
VALUES (3, 'Bancada 3', 'L-12', 30.0);

-- várias linhas de uma vez
INSERT INTO leituras (bancada_id, valor, instante)
VALUES (3, 24.8, 1793548800000),
       (3, 25.1, 1793548860000);

UPDATE bancadas
SET    limite = 32.0
WHERE  id = 3;

DELETE FROM leituras
WHERE  instante < 1790000000000;
```

DML — *Data Manipulation Language*

`INSERT`, `UPDATE`, `DELETE` e `SELECT`: trabalham com as linhas, não com a estrutura.

O erro clássico

`UPDATE` ou `DELETE` **sem** `WHERE` altera ou apaga **a tabela inteira**. Escreva o `WHERE` primeiro e teste-o com um `SELECT`.

Dica

Textos vão entre aspas simples (`'L-12'`); aspas duplas servem para nomes de colunas e tabelas.

SQL: consultas

SELECT: filtrando e ordenando

Consultas básicas

```sql
SELECT nome, sala
FROM   bancadas
WHERE  sala = 'L-12' AND limite >= 30
ORDER  BY nome ASC
LIMIT  10;

SELECT * FROM equipamentos
WHERE  descricao LIKE '%osciloscópio%';

SELECT * FROM leituras
WHERE  valor BETWEEN 20 AND 30;

SELECT * FROM bancadas WHERE id IN (1, 3, 5);

SELECT * FROM equipamentos
WHERE  bancada_id IS NULL;   -- nunca "= NULL"
```



Ordem lógica de execução

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Agregações e datas

Resumo por bancada

```
SELECT bancada_id,
       COUNT(*)              AS qtd,
       ROUND(AVG(valor), 1)  AS media,
       MAX(valor)            AS maxima
FROM   leituras
WHERE  instante >= :inicioDoDia
GROUP  BY bancada_id
HAVING MAX(valor) > 30
ORDER  BY media DESC;
```

Média por hora do dia

```
SELECT strftime('%H', instante / 1000,
                'unixepoch', 'localtime') AS hora,
       ROUND(AVG(valor), 1)               AS media
FROM   leituras
GROUP  BY hora
ORDER  BY hora;
```

Função	Resultado
`COUNT(*)`	número de linhas
`SUM`, `AVG`	soma e média
`MIN`, `MAX`	extremos
`ROUND(x, n)`	arredonda

`WHERE` *versus* `HAVING`

`WHERE` filtra **linhas** antes de agrupar; `HAVING` filtra **grupos** depois, e pode usar agregações.

Dica

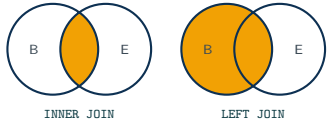
`:inicioDoDia` é um **parâmetro**: o app fornece o valor. É assim que o Room escreve consultas.

Juntando tabelas: JOIN

INNER e LEFT JOIN

```sql
-- só bancadas que têm equipamentos
SELECT b.nome, e.descricao
FROM   bancadas AS b
INNER  JOIN equipamentos AS e
       ON e.bancada_id = b.id;

-- todas as bancadas, mesmo sem equipamento
SELECT b.nome, COUNT(e.patrimonio) AS qtd
FROM   bancadas AS b
LEFT   JOIN equipamentos AS e
       ON e.bancada_id = b.id
GROUP  BY b.id;
```



Diferença

`INNER` devolve só as combinações que existem nas duas tabelas. `LEFT` mantém todas as linhas da tabela da esquerda e preenche com `NULL` o que faltar à direita.

Dica

Apelidos (`AS b`) deixam a consulta curta e evitam ambiguidade entre colunas de mesmo nome.

Relacionamento N:N e subconsultas

Agenda do dia (N:N)

```
SELECT u.nome AS usuario, b.nome AS bancada,
       r.inicio, r.fim
FROM   reservas AS r
JOIN   usuarios AS u ON u.id = r.usuario_id
JOIN   bancadas AS b ON b.id = r.bancada_id
WHERE  r.inicio >= :inicioDoDia
  AND  r.inicio <  :fimDoDia
ORDER  BY r.inicio;
```

Bancadas livres agora (subconsulta)

```
SELECT nome FROM bancadas
WHERE  id NOT IN (
  SELECT bancada_id FROM reservas
  WHERE  :agora BETWEEN inicio AND fim
);
```

Atravessando a tabela associativa

Para ir de usuários a bancadas, passa-se por `reservas`: um `JOIN` para cada FK.

Subconsulta

Um `SELECT` dentro de outro. Útil para "existe/não existe" (`IN`, `NOT IN`, `EXISTS`).

Atenção

`NOT IN` com uma subconsulta que devolve `NULL` retorna vazio. Prefira `NOT EXISTS` quando a coluna puder ser nula.

Transações e índices

Transações e ACID

Empréstimo de componente

```
BEGIN TRANSACTION;
  UPDATE estoque
  SET    qtd = qtd - 1
  WHERE  item = 'resistor-1k' AND qtd > 0;

  INSERT INTO emprestimos (item, usuario_id)
  VALUES ('resistor-1k', 7);
COMMIT;       -- ou ROLLBACK se algo falhar
```

Ou as duas operações acontecem, ou nenhuma. Sem transação, uma falha entre elas deixaria o estoque errado.

ACID	Garantia
Atomicidade	tudo ou nada
Consistência	as restrições valem antes e depois
Isolamento	transações simultâneas não se atrapalham
Durabilidade	após o `COMMIT`, sobrevive a queda de energia

No app

Room: `withTransaction { }`. Drift: `transaction(() async { })`. sqflite: `db.transaction`. Firestore: `runTransaction`. Milhares de inserções numa única transação são muito mais rápidas.

Índices e desempenho

Sem índice: varre a tabela inteira

```
EXPLAIN QUERY PLAN
SELECT * FROM leituras
WHERE  bancada_id = 3 AND instante > :desde;
-- SCAN leituras
```

Com índice composto: busca direta

```
CREATE INDEX idx_leituras_bancada_instante
  ON leituras (bancada_id, instante);
-- SEARCH leituras USING INDEX
--   idx_leituras_bancada_instante
--   (bancada_id=? AND instante>?)
```

Índice

Estrutura ordenada (árvore B) que aponta para as linhas, como o índice remissivo de um livro. Transforma uma varredura em uma busca.

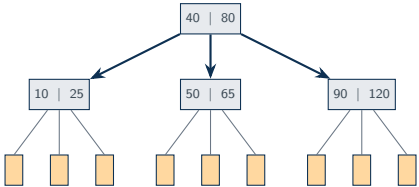
Quando criar

Colunas de `WHERE`, `JOIN` e `ORDER BY` frequentes; no composto, a de igualdade vem primeiro.

Atenção

Índices ocupam espaço e deixam `INSERT`/`UPDATE` mais lentos: crie só os usados.

Por dentro: árvore B e diário de escrita



folhas: páginas com as linhas (ou ponteiros)

Árvore B

Árvore balanceada em que cada nó é uma página com muitas chaves ordenadas. Com 100 chaves por página, três níveis já endereçam um milhão de linhas: poucas leituras de disco por busca.

Diário de escrita antecipada (WAL)

Antes de alterar o arquivo principal, o SGBD registra a mudança num diário. Se a energia cair no meio, o diário permite refazer ou descartar a operação: é assim que se garante a **durabilidade**.

WAL no SQLite

No modo WAL, leitores continuam lendo enquanto um escritor grava, e o diário é incorporado ao banco de tempos em tempos (*checkpoint*). O Room ativa esse modo por padrão em aparelhos com memória suficiente.

Bancos NoSQL

Modelando em documentos

Firestore: bancadas/b3

```
{
  "nome": "Bancada 3",
  "sala": "L-12",
  "limite": 30,
  "equipamentos": [
    { "patrimonio": "IFMA-0042",
      "descricao": "Osciloscópio" },
    { "patrimonio": "IFMA-0107",
      "descricao": "Fonte simétrica" }
  ],
  "ultimaLeitura": { "valor": 24.8,
                     "instante": "2026-11-01T15:00" }
}
// leituras: subcoleção bancadas/b3/leituras
```

Modelagem orientada a consultas

Em vez de normalizar, pergunta-se: **o que cada tela lê?** Os dados lidos juntos ficam no mesmo documento.

A lista curta de equipamentos fica **embutida** no documento da bancada; as leituras, que crescem sem limite, ficam numa **subcoleção**.

Atenção

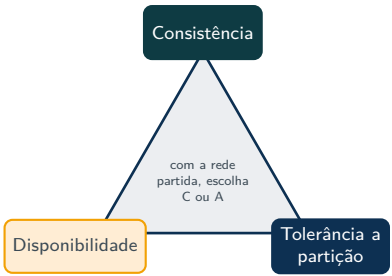
Sem *joins*: dados duplicados precisam ser atualizados em todos os lugares (Módulo 12).

SQL *versus* NoSQL

	Relacional (SQLite, PostgreSQL)	**Documentos (Firestore, MongoDB)**
Estrutura	esquema fixo, definido antes	flexível, por documento
Relacionamentos	chaves estrangeiras e *joins*	embutir ou duplicar dados
Consultas	SQL completo, ad hoc	limitadas, planejadas com índices
Integridade	garantida pelo banco (FK, `CHECK`)	garantida pelo app e pelas regras
Escala	vertical (máquina maior), réplicas	horizontal, distribuída
Tempo real e offline	não nativo	nativo (Firestore)

Não é "um ou outro": o LabMonitor usa **SQLite no aparelho** como fonte da verdade local e **Firestore na nuvem** para compartilhar reservas em tempo real.

Bancos distribuídos: CAP, ACID e BASE



Teorema CAP

Num sistema distribuído, quando a rede se divide (partição), não é possível garantir ao mesmo tempo **consistência** (todos leem o dado mais recente) e **disponibilidade** (toda requisição recebe resposta). Proposto por Eric Brewer (2000) e demonstrado em 2002.

BASE e consistência eventual

Alternativa ao ACID em sistemas distribuídos: sempre responder, aceitando que as réplicas fiquem iguais **depois** de um tempo.

No LabMonitor

Uma reserva feita offline no Firestore aparece primeiro só no aparelho e é sincronizada quando a rede volta; conflitos precisam de uma regra (por exemplo, a última escrita vence).

Segurança e prática

SQL injection

ERRADO: concatenar texto digitado

```
val sql = "SELECT * FROM usuarios " +
          "WHERE email = '$email'"
db.rawQuery(sql, null)
// email digitado:  ' OR '1'='1
// vira: ... WHERE email = '' OR '1'='1'
//       -> devolve TODOS os usuários
```

CERTO: parâmetros

```
db.rawQuery(
  "SELECT * FROM usuarios WHERE email = ?",
  arrayOf(email))

// Room: parâmetros sempre seguros
@Query("SELECT * FROM usuarios WHERE email = :email")
suspend fun porEmail(email: String): Usuario?
```

O ataque

Texto do usuário colado no SQL pode mudar o comando: ler dados de outros, apagar tabelas, burlar o login.

A defesa

Use sempre **parâmetros** (`?` ou `:nome`). O banco trata o valor como dado, nunca como código.

Dart (sqflite)

```
db.rawQuery(
    'SELECT * FROM usuarios WHERE email = ?',
      [email]);
```

Praticando com SQLite

sqlite3 no terminal

```
sqlite3 labmonitor.db
sqlite> .read labmonitor.sql     -- executa o script
sqlite> .tables                  -- lista as tabelas
sqlite> .schema leituras         -- mostra a estrutura
sqlite> .headers on
sqlite> .mode column
sqlite> SELECT * FROM bancadas;
sqlite> .quit
```

Dica

No Linux: `sudo apt install sqlite3`. No Windows e no macOS, baixe em `https://sqlite.org/download.html`.

Ferramentas gráficas

- **DB Browser for SQLite**: gratuito, cria tabelas, edita dados e roda SQL.
- **Database Inspector** do Android Studio: banco do app *em execução*, ao vivo (Módulo 10).
- Extensões de SQLite para o VS Code.

SQL	Room	Drift
`CREATE TABLE`	`@Entity`	`extends Table`
`PRIMARY KEY`	`@PrimaryKey`	`autoIncrement()`
`REFERENCES`	`ForeignKey`	`references()`
`CREATE INDEX`	`Index`	`@TableIndex`
`SELECT ... ?`	`@Query`	`select()`
`BEGIN ... COMMIT`	`@Transaction`	`transaction()`

Fechamento

Resumo

Resumo

- Um SGBD organiza, protege e consulta dados; no celular, o SQLite embutido é a base do Room e do Drift.
- Modele com entidades, atributos e relacionamentos; 1:N vira FK, N:N vira tabela associativa.
- Normalize até a 3FN para guardar cada fato uma vez; desnormalize só com motivo medido.
- DDL define a estrutura; DML manipula linhas. Nunca `UPDATE`/`DELETE` sem `WHERE`.
- `SELECT` com `WHERE`, `GROUP BY`, `HAVING`, `JOIN` e subconsultas responde às perguntas do app.
- Transações garantem ACID (com diário de escrita); índices em árvore B aceleram buscas; parâmetros evitam *SQL injection*.
- Bancos de documentos modelam pelas consultas; em sistemas distribuídos, o CAP obriga a escolher entre consistência e disponibilidade.

Glossário (1/2)

Termo	Significado
SGBD	software que armazena, consulta e protege os dados
Entidade / atributo	coisa sobre a qual se guardam dados / característica dela
Chave primária (PK)	coluna que identifica cada linha de forma única
Chave estrangeira (FK)	coluna que aponta para a PK de outra tabela
Normalização	organizar tabelas para guardar cada fato uma única vez
Dependência funcional	quando o valor de uma coluna determina o de outra
Álgebra relacional	operações sobre tabelas (seleção, projeção, junção) por trás do SQL
DDL / DML	comandos que definem a estrutura / manipulam as linhas

Glossário (2/2)

Termo	Significado
Transação	conjunto de operações executadas como uma unidade (tudo ou nada)
ACID	atomicidade, consistência, isolamento e durabilidade
Índice	estrutura ordenada (árvore B) que acelera buscas
WAL	diário de escrita antecipada que garante durabilidade
Plano de consulta	estratégia escolhida pelo otimizador para executar o SQL
Teorema CAP	com partição de rede, escolhe-se consistência ou disponibilidade
Consistência eventual	réplicas ficam iguais depois de algum tempo
SQL *injection*	ataque que insere SQL em dados concatenados ao comando

Questões de revisão

1. Por que usar um SGBD em vez de gravar as leituras num arquivo de texto?
2. Como um relacionamento N:N entre usuários e bancadas vira tabelas?
3. Que anomalias a "planilha única" provoca, e qual forma normal as resolve?
4. Escreva em álgebra relacional e em SQL: patrimônio dos equipamentos da Bancada 3.
5. O que garante a durabilidade de uma transação se a energia cair logo após o `COMMIT`?
6. Por que um índice composto (`bancada_id`, `instante`) acelera a consulta das últimas leituras?
7. Explique o teorema CAP com o exemplo de uma reserva feita offline.
8. Mostre por que concatenar o e-mail digitado no SQL é perigoso e como corrigir.

Exercícios

Exercício 1 — Modelagem

Acrescente ao diagrama ER o **empréstimo de componentes**: um usuário pega vários componentes, cada componente tem estoque e cada empréstimo tem data de retirada e de devolução. Converta para tabelas.

Exercício 2 — Normalização

Normalize até a 3FN uma planilha de aulas práticas com as colunas: data, turma, professor, e-mail do professor, laboratório, bancadas usadas (lista).

Exercício 3 — Consultas

Escreva SQL para: (a) as 3 bancadas com mais alertas no mês; (b) usuários sem nenhuma reserva; (c) a média diária de temperatura da Bancada 3 na última semana; (d) equipamentos de bancadas reservadas agora.

LabMonitor — Etapa 9

Entregue o diagrama ER e o script `labmonitor.sql` (tabelas, índices, 20 linhas de exemplo e 10 consultas comentadas), testado no DB Browser for SQLite. Ele será a base das tabelas Room ou Drift no Módulo 10.

Para aprofundar

- Linguagem SQL do SQLite: `https://www.sqlite.org/lang.html`
- Tipos de dados no SQLite: `https://www.sqlite.org/datatype3.html`
- Chaves estrangeiras no SQLite: `https://www.sqlite.org/foreignkeys.html`
- `EXPLAIN QUERY PLAN`: `https://www.sqlite.org/eqp.html`
- DB Browser for SQLite: `https://sqlitebrowser.org`
- Modelo de dados do Firestore: `https://firebase.google.com/docs/firestore/data-model`
- OWASP — SQL injection: `https://owasp.org/www-community/attacks/SQL_Injection`
- Livro clássico: Elmasri e Navathe, *Sistemas de Banco de Dados* (Pearson).

Respostas das questões de revisão

1. Busca rápida com índices, integridade (chaves e restrições), sem duplicação, transações que não corrompem os dados e consultas flexíveis em SQL.
2. Com uma tabela associativa (`reservas`) que tem uma FK para `usuarios` e outra para `bancadas`, além dos próprios atributos (início, fim).
3. Anomalias de inserção, atualização e exclusão causadas por dados repetidos. A 3FN (com 1FN e 2FN) guarda cada fato uma vez, em tabelas separadas.
4. $\pi_{\text{patrimonio}}(\sigma_{\text{bancada_id}=3}(\text{equipamentos}))$; `SELECT patrimonio FROM equipamentos WHERE bancada_id = 3;`
5. O diário de escrita (WAL ou *journal*): a mudança é registrada antes de ser aplicada e pode ser refeita na reinicialização.
6. A árvore B ordena primeiro pela bancada e depois pelo instante: o banco vai direto à faixa da bancada 3 e percorre só as leituras mais recentes, sem varrer a tabela.
7. Sem rede (partição), o Firestore escolhe disponibilidade: aceita a reserva localmente; a consistência entre aparelhos só vem quando a sincronização acontece.
8. O texto digitado pode conter SQL (`' OR '1'='1`) e mudar o comando. Correção: parâmetros (`?` ou `:email`), que tratam o valor como dado.

Fim do Módulo 09

Próximo: Módulo 10 — Persistência local: Room e DataStore (trilha Kotlin) ou Drift, sqflite e shared_preferences (trilha Flutter).